LINUX

COMPLETE TEXTBOOK

From Architecture to DevOps

16 Chapters • Real-World Analogies • Command Reference

tech__.talks

Chapter 1: Introduction to Linux

Linux is a free, open-source operating system that powers everything from smartphones and web servers to supercomputers. Created by Linus Torvalds in 1991, it has grown into the world's most widely used OS in enterprise environments.

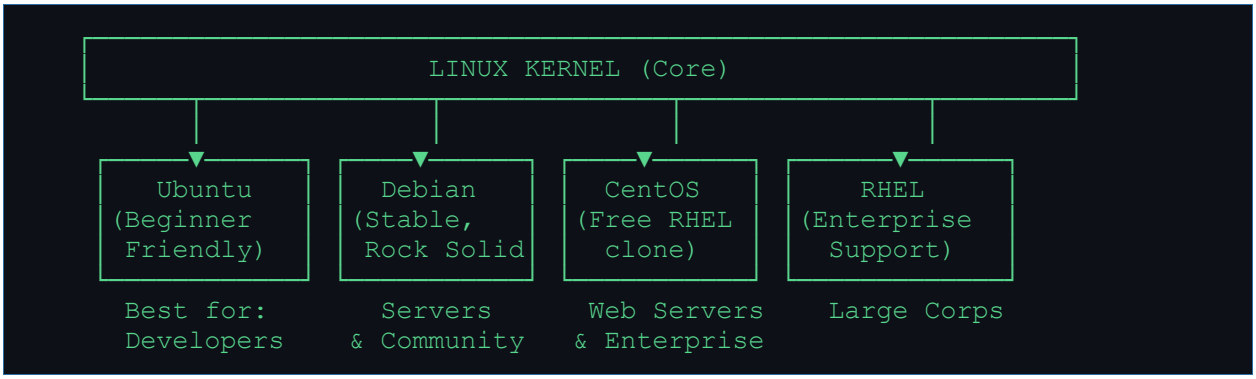
> **Real-World Analogy: Open Source**
> *Think of Windows like a house you rent — you can live in it but you can't modify the walls. Linux is like owning the house — you can rebuild every room, repaint everything, and share the blueprint with your neighbors for free!*

Key Characteristics

Command	What it does	Real-life Analogy
`Open-source`	Code is publicly visible and editable	*Like a public recipe anyone can improve*
`Multi-user`	Many users can log in simultaneously	*Like a hotel with 500 guests at once*
`Multi-tasking`	Runs many programs at the same time	*Like a chef cooking 10 dishes at once*
`CLI-focused`	Controlled via text commands	*Like texting your computer instructions*
`Kernel-based`	Core engine manages all hardware	*Like an engine inside a car*

Popular Linux Distributions

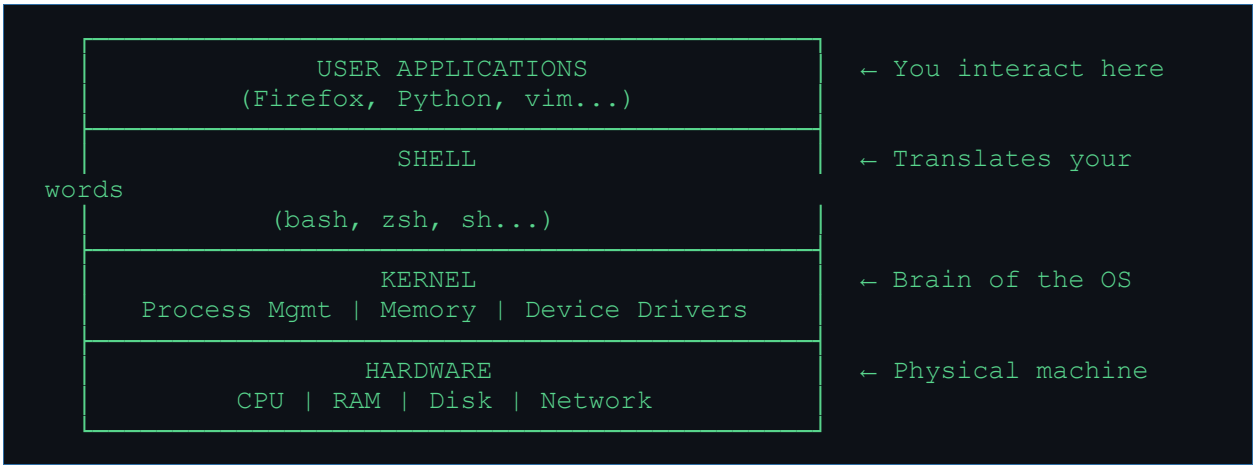
A distribution (distro) is Linux packaged with extra tools and a desktop environment. Think of it as different car brands using the same engine.



> Quick Tip: Ubuntu is the best starting point for beginners. It has the largest community and most tutorials online.

Chapter 2: Linux Architecture

Linux is built in layers, like an onion. Each layer talks to the layer below it. Understanding this helps you troubleshoot problems faster.



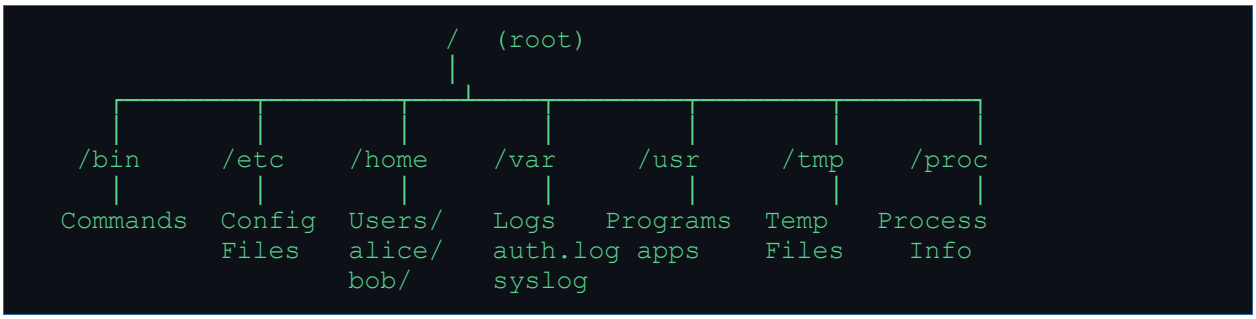
> **Real-World Analogy: Office Building**
> *Imagine a company building. The hardware is the physical office. The kernel is the HR + facilities team managing resources. The shell is the receptionist who takes your request and routes it. You (the user) sit at the top giving instructions. You never talk directly to the janitor — the receptionist handles that!*

Kernel Responsibilities

Command	What it does	Real-life Analogy
`Process Mgmt`	Runs & schedules programs	*Like a traffic cop managing cars*
`Memory Mgmt`	Allocates RAM to programs	*Like a hotel room allocation system*
`Device Drivers`	Talks to hardware devices	*Like a translator between you and a foreigner*
`File System`	Manages how data is stored	*Like a librarian organizing all books*

Chapter 3: Linux File System Structure

In Linux, everything is a file — even hardware devices! The file system starts from a single root directory called / (forward slash). All other directories branch out from here like a tree.



Important Directories Explained

Command	What it does	Real-life Analogy
/bin	Essential commands (ls, cp, mv)	*Like tools in a kitchen drawer*
/boot	Boot loader & kernel files	*Like the ignition system of a car*
/etc	All configuration files	*Like the settings panel of your phone*
/home	Personal folders for each user	*Like everyone's private bedroom*
/root	Root user's home directory	*Like the CEO's private office*
/var	Logs, emails, variable data	*Like a filing cabinet with daily reports*
/usr	Installed user programs	*Like the Apps folder on your phone*
/tmp	Temporary files (deleted on reboot)	*Like a whiteboard — cleared every day*
/proc	Live process information (virtual)	*Like a dashboard showing live car stats*

Real-World Analogy: Company Office

Think of / as the company headquarters. /home is the employees' personal desks. /etc is the HR policy folder. /var is the manager's daily report drawer. /bin is the company toolbox. /tmp is the sticky-note board — throw it away each morning!

Quick Tip: Use 'ls -la /' to see all root directories. The 'd' in front of drwxr-xr-x means directory!

Chapter 4: File & Directory Management

These are your day-to-day survival commands. Master these first — they are used in every Linux task you'll ever do.

Command	What it does	Real-life Analogy
`pwd`	Print current location (where am I?)	*Like asking 'what floor am I on?'*
`ls`	List files and folders	*Like opening a folder on your desktop*
`ls -la`	List with details + hidden files	*Like opening folder with full file details*
`cd /path`	Move into a directory	*Like walking into a room*
`cd ..`	Go back one level	*Like stepping out of a room*
`mkdir name`	Create a new directory	*Like creating a new folder*
`rm -rf dir`	Delete folder and everything inside	*Like sending a folder to shredder*
`cp src dst`	Copy file from one place to another	*Like photocopying a document*
`mv src dst`	Move or rename a file	*Like picking up and placing a box*
`touch file`	Create an empty file	*Like creating a blank notepad*
`cat file`	Print file contents to screen	*Like reading a document out loud*
`less file`	View file one page at a time	*Like reading a book page by page*

Practical Example Session

```
# 1. Find out where you are
$ pwd
/home/alice

# 2. List all files including hidden
$ ls -la
drwxr-xr-x  alice  alice  4096  /home/alice
-rw-r--r--  alice  alice   220  .bashrc

# 3. Create a project folder
$ mkdir myproject
$ cd myproject

# 4. Create a file and view it
$ echo "Hello Linux!" > notes.txt
$ cat notes.txt
Hello Linux!
```

```
# 5. Copy, rename, and delete
$ cp notes.txt backup.txt
$ mv notes.txt main_notes.txt
$ rm backup.txt
```

Real-World Analogy: Physical Office

cd = walking through hallways. mkdir = building a new room. cp = photocopying a document. mv = physically picking up a file and putting it in a different drawer. rm -rf = throwing the entire drawer + contents into a shredder. BE CAREFUL with rm -rf — Linux won't ask 'Are you sure?'!

Chapter 5: File Permissions & Ownership

Linux security is built on permissions. Every file has an owner and rules about who can read, write, or execute it. This is your first line of security defense.

```
Permission string:  -rwxr-xr--
                    |||||||||└── Other: Read only
                    ||||||||└─── Other: No write
                    |||||||└──── Other: No execute
                    ||||||└───── Group: Read
                    |||||└────── Group: No write
                    ||||└─────── Group: Execute
                    |||└──────── Owner: Read
                    ||└───────── Owner: Write + Execute

- = regular file    d = directory    l = symbolic link

Permission Values:  r=4   w=2   x=1
rwx = 4+2+1 = 7     r-x = 4+0+1 = 5     r-- = 4+0+0 = 4
```

Real-World Analogy: House Keys

A file is like a safe. r (read) = you can look inside. w (write) = you can put things in or take out. x (execute) = for a program, it means you can run it (like being allowed to start a car). chmod 755 = owner can do everything, group and others can only read and run. chmod 600 = only YOU can read and write — like a personal diary with a lock!

Permission Commands

Command	What it does	Real-life Analogy
`chmod 755 file`	Set rwxr-xr-x permissions	*Give owner full access, others read+run*
`chmod 600 file`	Set rw------- permissions	*Private file, only owner reads/writes*
`chmod +x file`	Add execute to everyone	*Make a script runnable*
`chown alice file`	Change owner to alice	*Transfer file ownership*
`chown alice:devs file`	Change owner + group	*Give to alice, group devs*
`chgrp devs file`	Change group only	*Move file to devs group*

```
# Check permissions
$ ls -l script.sh
-rw-r--r-- 1 alice devs 1024 Jan 01 script.sh

# Make it executable
$ chmod +x script.sh
$ ls -l script.sh
-rwxr-xr-x 1 alice devs 1024 Jan 01 script.sh

# Lock a private key file
```

```
$ chmod 600 ~/.ssh/id_rsa

# Change owner and group
$ chown alice:devs report.txt
```

 Quick Tip: SSH private key files MUST be chmod 600 — SSH will refuse to work if they're too open!

Chapter 6: User & Group Management

Linux supports multiple users simultaneously. Users are organized into groups to manage permissions efficiently. The root user is the all-powerful administrator.

> **Real-World Analogy: Company HR System**
>
> *Imagine a company. The root user is the CEO — can do anything. Regular users are employees. Groups are departments (devs, finance, HR). When you give a file access to the 'devs' group, all developers automatically get that access — just like giving a key card to an entire department!*

Command	What it does	Real-life Analogy
`useradd alice`	Create a new user called alice	*Like creating a new employee account*
`passwd alice`	Set/change alice's password	*Like resetting someone's login*
`usermod -aG devs alice`	Add alice to the devs group	*Like adding someone to a department*
`userdel alice`	Delete user alice	*Like removing an employee account*
`groupadd devs`	Create a new group	*Like creating a new department*
`groups alice`	Show all groups alice belongs to	*Like checking employee's team memberships*
`sudo command`	Run a command as root	*Like using the master key for one task*
`su - alice`	Switch to user alice	*Like logging in as a different employee*

Important System Files

```
/etc/passwd  — List of all users
Format: username:x:UID:GID:comment:home:shell
alice:x:1001:1001:Alice Smith:/home/alice:/bin/bash
       |    |    |
       |    |    └── Group ID
       |    └─────── User ID (UID)
       └──────────── 'x' means password is in /etc/shadow

/etc/shadow — Encrypted passwords (only root can read!)
/etc/group  — Group definitions and memberships
```

```
# Full workflow: Create user, set password, add to group
$ useradd -m -s /bin/bash alice      # -m creates home dir
$ passwd alice                        # set password
$ groupadd developers                 # create group
$ usermod -aG developers alice        # add to group
```

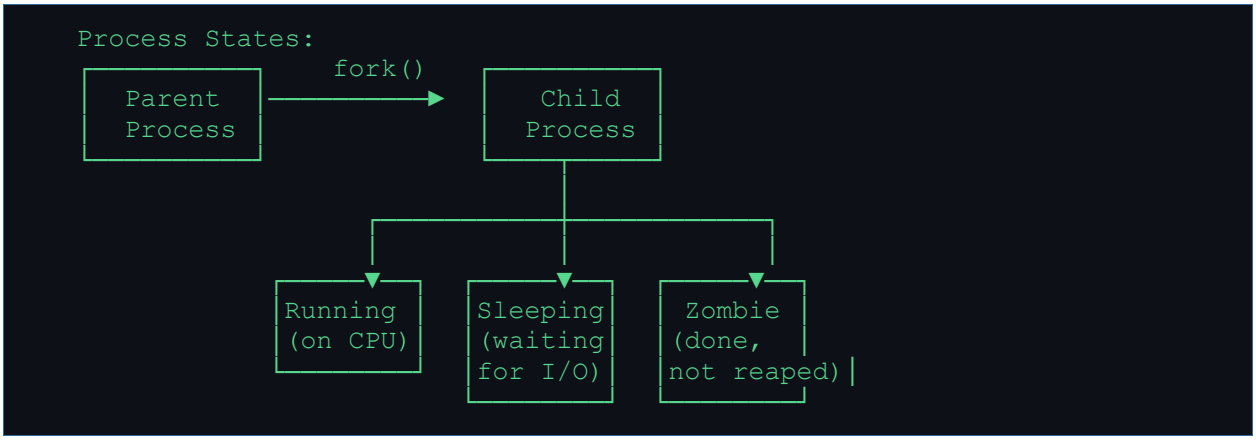
```
$ groups alice                          # verify
alice : alice developers
```

Chapter 7: Process Management

Every program running on Linux is a process. Each gets a unique Process ID (PID). Understanding processes helps you find what's eating your CPU or RAM and stop it.

Real-World Analogy: Restaurant Kitchen

Imagine a busy kitchen. Each chef (process) has an employee number (PID). The head chef (init/systemd, PID=1) started all the other chefs. If a chef is stuck (zombie process), they're finished cooking but waiting for the manager to take their dish. kill -9 PID = firing a chef immediately, no questions asked!



Command	What it does	Real-life Analogy
`ps -ef`	Show all running processes	*Like viewing all open apps*
`ps aux | grep nginx`	Find specific process	*Like searching for one app by name*
`top`	Live process monitor	*Like Task Manager in Windows*
`htop`	Better interactive process viewer	*Like Task Manager with colors*
`kill 1234`	Stop process with PID 1234	*Like gently closing an app*
`kill -9 1234`	Force-kill process 1234	*Like force-quitting a frozen app*
`nice -n 10 cmd`	Start command with lower priority	*Like asking someone to work slower*
`bg`	Send current job to background	*Like minimizing a window*
`fg`	Bring background job to front	*Like maximizing a minimized window*
`jobs`	List all background jobs	*Like seeing your minimized windows*

```
# Find and kill a frozen process
$ ps aux | grep apache2
alice    5432  99.9  1.2  apache2 ...
```

```
$ kill 5432          # Try graceful shutdown first
$ kill -9 5432       # Force kill if still running

# Run something in background
$ python3 server.py &         # & sends to background
[1] 5567                      # Job 1, PID 5567
$ jobs                        # check background jobs
[1]+  Running   python3 server.py
```

Chapter 8: Memory & Disk Management

Linux manages memory and disk space automatically, but you need to monitor them to prevent servers from running out. Full disks crash servers. Low RAM slows them down.

Real-World Analogy: Office Space

RAM is your desk space — the bigger it is, the more papers you can work on at once. Disk is your filing cabinet — stores everything long-term. Swap memory is an extra table in the hallway — slower than your desk, used when desk is full. Inodes are like the index cards in your filing cabinet — you can run out of index cards even if the cabinet has space!

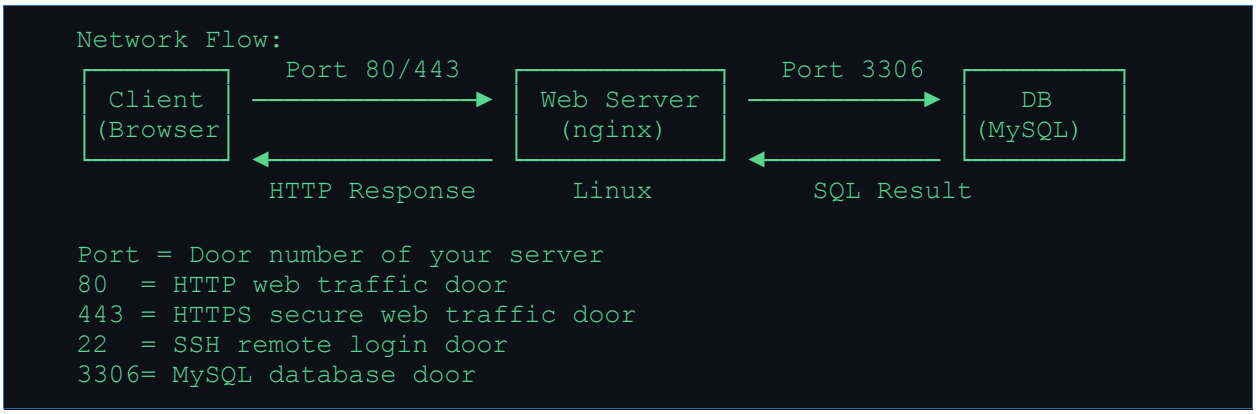
Command	What it does	Real-life Analogy
`free -m`	Show RAM and swap usage in MB	*Like checking how full your desk is*
`df -h`	Show disk usage per partition	*Like checking filing cabinet fullness*
`du -sh /var`	Show size of a directory	*Like weighing a specific drawer*
`lsblk`	List all storage devices	*Like listing all storage boxes*
`mount /dev/sdb /mnt/data`	Attach a disk to a folder	*Like plugging in a USB drive*
`umount /mnt/data`	Safely detach a disk	*Like safely ejecting a USB drive*
`fdisk /dev/sdb`	Partition a disk	*Like dividing a cabinet into sections*

```
# Quick health check of a server
$ free -m
              total        used        free      shared  buff/cache
Mem:          16000        8200        2300         450        5500
Swap:          4000         200        3800

# Check if disk is full (CRITICAL: watch for 100%)
$ df -h
Filesystem      Size  Used Avail Use%  Mounted on
/dev/sda1        50G   43G  4.2G  92%  /         ← Getting full!
/dev/sdb1       200G   50G  150G  25%  /data

# Find what's eating disk space
$ du -sh /var/log/*
4.5G    /var/log/nginx
256M    /var/log/syslog
```

Quick Tip: If df -h shows 100% disk usage, find large files with: find / -size +500M -type f 2>/dev/null

Chapter 9: Networking in Linux

Linux is built for networking. Every server, cloud instance, and container uses Linux networking. Understanding these commands is critical for DevOps and sysadmin roles.



Command	What it does	Real-life Analogy
`ip a`	Show IP addresses of all interfaces	*Like checking your phone's IP address*
`ifconfig`	Old way to show network info	*Like the older network settings panel*
`ping google.com`	Test if a server is reachable	*Like calling someone to see if they answer*
`netstat -tulpn`	Show which ports are listening	*Like checking which doors are open*
`ss -tulpn`	Modern replacement for netstat	*Like a faster version of netstat*
`curl http://example.com`	Fetch a URL and show response	*Like a browser without the visuals*
`wget http://site.com/file`	Download a file from internet	*Like clicking 'Download' in a browser*
`scp file user@host:/path`	Securely copy file to remote server	*Like email-attaching a file to a server*
`ssh alice@192.168.1.10`	Login to remote server securely	*Like remote desktop but via terminal*

```
# Check your server's IP
$ ip a | grep inet
inet 192.168.1.100/24    # your private IP
inet 34.22.11.5/32       # your public IP (if on cloud)

# Check what's running on port 80
$ ss -tulpn | grep :80
tcp  LISTEN  nginx   0.0.0.0:80

# Test connectivity
$ ping -c 4 8.8.8.8          # ping 4 times
```

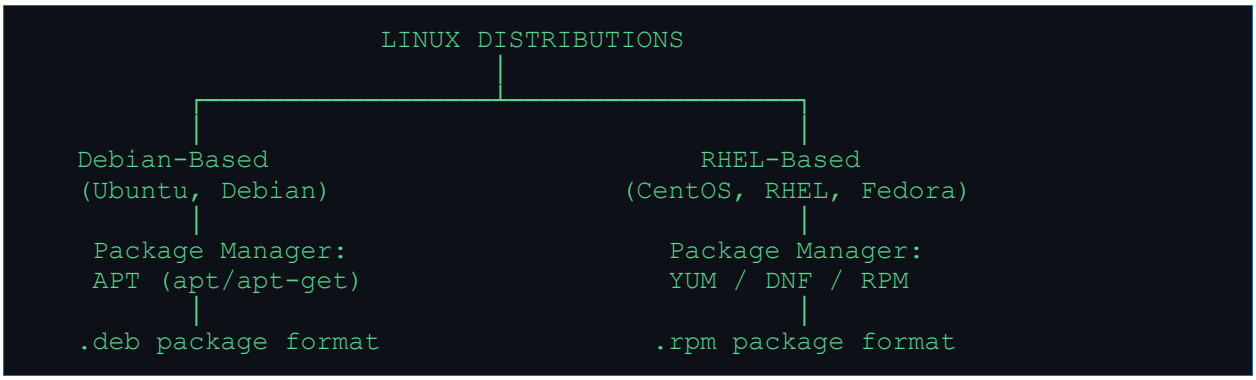
```
$ curl -I https://google.com  # check HTTP headers

# Securely copy a file to another server
$ scp report.csv alice@server2:/home/alice/
$ ssh alice@server2           # login remotely
```

Real-World Analogy: Ports are Doors

Your server is a building. Each port is a numbered door. Port 22 is the staff entrance (SSH). Port 80 is the main public entrance (HTTP). Port 443 is the secure entrance with a guard (HTTPS). netstat/ss lets you see which doors are open and who's standing behind each one. A firewall is the security guard deciding who can knock on which door.

Chapter 10: Package Management

Installing software on Linux is done through package managers — tools that download, install, update, and remove software automatically. Different Linux families use different package managers.



Debian/Ubuntu — APT Commands

Command	What it does	Real-life Analogy
`apt update`	Refresh the list of available packages	*Like refreshing the App Store catalog*
`apt upgrade`	Install all available updates	*Like clicking 'Update All' on your phone*
`apt install nginx`	Install nginx web server	*Like clicking 'Install' in App Store*
`apt remove nginx`	Remove nginx	*Like uninstalling an app*
`apt autoremove`	Remove unused dependencies	*Like clearing app leftovers*
`apt search redis`	Search for a package by name	*Like searching the App Store*

RHEL/CentOS — YUM/DNF Commands

Command	What it does	Real-life Analogy
`yum install httpd`	Install Apache web server	*Same as apt install on Ubuntu*
`dnf install httpd`	Newer version of yum install	*dnf is yum's faster successor*
`yum update`	Update all packages	*Like apt upgrade on Ubuntu*
`rpm -ivh pkg.rpm`	Install a local .rpm file	*Like installing a downloaded .exe*
`rpm -qa`	List all installed packages	*Like viewing all installed apps*

Real-World Analogy: App Store

apt/yum is like your phone's App Store. apt update = refresh the app list. apt install = tap Install. apt remove = delete an app. The package manager handles all dependencies automatically — just like installing WhatsApp also installs required system libraries without you knowing!

 Quick Tip: Always run 'apt update' BEFORE 'apt install'. Without updating, you might install an outdated version!

Chapter 11: Shell & Bash Scripting

Bash scripting lets you automate repetitive tasks. Instead of typing 50 commands every morning, write a script that does it in 1 click. This is a superpower for DevOps engineers.

> **Real-World Analogy: Recipe Automation**
>
> *Imagine you make the same sandwich every day. Instead of doing each step manually (get bread, butter, slice, stack...), you write a recipe card once and follow it every time. A bash script is your recipe card — written once, executed perfectly every time by the computer!*

Script Structure

```
#!/bin/bash
# ↑ Shebang: tells system to use bash to run this script

# 1. VARIABLES
NAME="Alice"
AGE=25
echo "Hello, $NAME! You are $AGE years old."

# 2. USER INPUT
echo "Enter your name:"
read USERNAME
echo "Welcome, $USERNAME!"

# 3. IF/ELSE CONDITIONS
if [ $AGE -ge 18 ]; then
    echo "Adult"
elif [ $AGE -ge 13 ]; then
    echo "Teenager"
else
    echo "Child"
fi

# 4. FOR LOOP
for SERVER in web01 web02 web03; do
    echo "Checking $SERVER..."
    ping -c 1 $SERVER
done

# 5. WHILE LOOP
COUNT=1
while [ $COUNT -le 5 ]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done

# 6. FUNCTION
greet_user() {
    local user=$1       # $1 = first argument
    echo "Hello, $user!"
}
greet_user "Bob"        # call the function
```

Input/Output Redirection & Pipes

Command	What it does	Real-life Analogy
`cmd > file.txt`	Write output to file (overwrite)	*Like saving console output to a file*
`cmd >> file.txt`	Append output to file	*Like adding more lines to a file*
`cmd < input.txt`	Use file as input to command	*Like feeding a file into a command*
`cmd1 | cmd2`	Send output of cmd1 into cmd2	*Like assembly line — each step processes*
`cmd 2> err.txt`	Redirect errors to file	*Like saving only error messages*
`cmd &> all.txt`	Save both output and errors	*Like recording everything*

```
# Powerful pipe examples
$ cat /var/log/nginx/access.log | grep "404" | wc -l
# Count all 404 errors in nginx log

$ ps aux | grep nginx | awk '{print $2}'
# Get PID of nginx process

$ cat /etc/passwd | cut -d: -f1 | sort
# List all usernames alphabetically
```

Real-World Analogy: Pipes

Think of a pipe | as a conveyor belt in a factory. First worker (grep) picks out the 404 errors from the log. Passes them on the belt to the next worker (wc -l) who counts them. Each command does one small job, and pipes connect them into a powerful assembly line!

Chapter 12: System Services & systemd

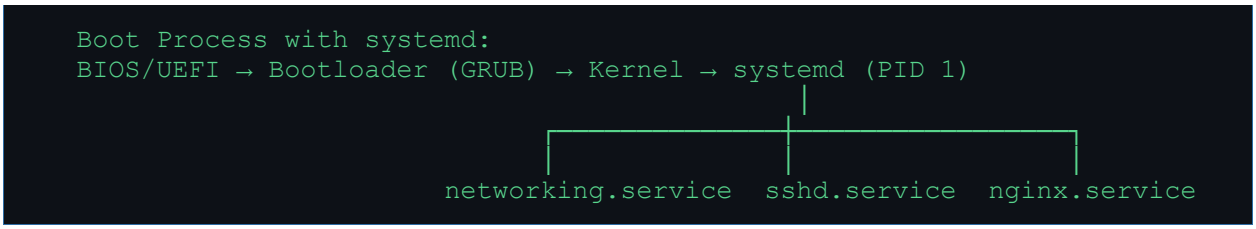
Services are background programs that run automatically and keep your server working. A web server, database, or SSH daemon are all services. systemd is the system that manages all of them.

Real-World Analogy: Building Services

Think of systemd as the building manager. Services are like the building's systems — electricity, water, elevator, security. The building manager (systemd) starts them all when the building opens (boots), monitors them, restarts them if they fail, and shuts them down properly at end of day. You just tell the manager what you need — systemctl is your phone call to the building manager!

Command	What it does	Real-life Analogy
`systemctl start nginx`	Start the nginx service now	*Like turning on the A/C*
`systemctl stop nginx`	Stop nginx gracefully	*Like turning off the A/C properly*
`systemctl restart nginx`	Stop and start nginx	*Like rebooting a stuck elevator*
`systemctl reload nginx`	Reload config without stopping	*Like refreshing settings without restart*
`systemctl status nginx`	Check if nginx is running	*Like checking if A/C is on*
`systemctl enable nginx`	Auto-start nginx at boot	*Like setting A/C to auto-start at 9am*
`systemctl disable nginx`	Don't auto-start at boot	*Like removing the auto-start schedule*
`systemctl list-units`	Show all services and their state	*Like a dashboard of all building systems*

```
# Full service management workflow
$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/lib/systemd/system/nginx.service)
   Active: active (running) since Mon 2024-01-01 10:00:00 UTC ✓

$ systemctl stop nginx
$ systemctl status nginx
   Active: inactive (dead)

$ systemctl start nginx
$ systemctl enable nginx   # survive reboots
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service
```



```
                        (network comes up)   (SSH starts)   (web starts)

Boot Targets (like runlevels):
multi-user.target = servers (no GUI) | graphical.target = desktop
```

Chapter 13: Log Management

Logs are the black box recorder of your Linux system. When something breaks at 3am, logs tell you exactly what happened. Every sysadmin and DevOps engineer must master log reading.

Command	What it does	Real-life Analogy
`/var/log/syslog`	General system activity log	*Like the main diary of the server*
`/var/log/messages`	General messages (RHEL/CentOS)	*Same as syslog on Red Hat systems*
`/var/log/auth.log`	All login attempts & sudo usage	*Like security camera footage*
`/var/log/nginx/`	Nginx web server access & errors	*Like a visitor log for your website*
`/var/log/kern.log`	Kernel messages (hardware issues)	*Like hardware error reports*
`/var/log/cron`	Scheduled task (cron job) logs	*Log of automated task runs*

Log Reading Commands

Command	What it does	Real-life Analogy
`tail -f /var/log/syslog`	Live-stream logs as they happen	*Like watching live CCTV footage*
`tail -100 /var/log/syslog`	Show last 100 lines	*Like reading the last page of a diary*
`grep 'ERROR' /var/log/app.log`	Search logs for errors	*Like ctrl+F in a document*
`journalctl -u nginx`	Show logs for nginx service	*Service-specific log viewer*
`journalctl -f`	Live stream of all system logs	*Like tail -f but more powerful*
`journalctl --since '1 hour ago'`	Show logs from last hour	*Like filtering by time*

```
# Troubleshoot a failing nginx
$ systemctl status nginx          # check if it's running
$ journalctl -u nginx -n 50       # last 50 nginx log lines
$ tail -f /var/log/nginx/error.log # live error stream

# Find failed SSH logins (security check)
$ grep "Failed password" /var/log/auth.log | tail -20

# See who logged in recently
$ last -n 20

# Check disk-related kernel errors
$ dmesg | grep -i error | tail -20
```

Real-World Analogy: Security Camera

Logs are CCTV footage for your server. tail -f is like watching live footage. grep 'Failed password' in auth.log is like fast-forwarding to find intruder attempts. journalctl is a smart DVR that lets you filter by time, service, or severity. ALWAYS check logs first when something breaks — the answer is almost always there!

Chapter 14: Environment Variables

Environment variables are named values stored in your shell session. They configure how programs behave — like global settings for your terminal. PATH is the most important one.

Real-World Analogy: GPS Settings

Imagine your phone's GPS settings. You set your home address once (HOME), favorite route preference (PATH), preferred language (LANG). Every app uses these settings without you having to repeat them. Environment variables work the same way — set once, used by all programs automatically!

```
# View all environment variables
$ env
$ printenv

# View a specific variable
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin

# PATH = where Linux looks for commands when you type them
# If 'myapp' is in /opt/myapp/bin, add it to PATH:
$ export PATH=$PATH:/opt/myapp/bin

# Set a temporary variable (only this session)
$ MY_VAR="hello"
$ echo $MY_VAR
hello

# Set a permanent variable (survives logout)
$ echo 'export MY_VAR="hello"' >> ~/.bashrc
$ source ~/.bashrc    # reload without logging out
```

Important Config Files

Command	What it does	Real-life Analogy
`~/.bashrc`	Runs for every new terminal session	*Like autorun settings for your terminal*
`~/.profile`	Runs at login (for login shells)	*Loaded when you SSH in or login*
`/etc/environment`	System-wide environment variables	*Global settings for all users*
`/etc/profile`	System-wide profile for all users	*Like company-wide terminal settings*

Chapter 15: Linux Security

Security is not optional — it is the foundation. A Linux server exposed to the internet is attacked within minutes of going online. These are your essential defenses.

SSH Key Authentication

```
   Password Login (Weak):                SSH Key Login (Strong):
   Client ──password──► Server           Client ──► Server
   (can be guessed,                      Uses 2 keys:
    brute-forced,                        □ Private Key: stays on YOUR
computer
    intercepted)                         □ Public Key: stored on server

   Like: Lock + Key system
   Public key = padlock (give to anyone)
   Private key = the key (NEVER share)
   Server has your padlock. Only your key opens it!
```

```
# Generate SSH key pair
$ ssh-keygen -t ed25519 -C "alice@mycompany.com"
# Creates:  ~/.ssh/id_ed25519      (private - NEVER share!)
#           ~/.ssh/id_ed25519.pub  (public - share with servers)

# Copy public key to a server
$ ssh-copy-id alice@server.example.com

# Login without password (key auth)
$ ssh alice@server.example.com

# Disable password login (server-side hardening)
$ sudo nano /etc/ssh/sshd_config
# Set: PasswordAuthentication no
# Set: PermitRootLogin no
$ sudo systemctl restart sshd
```

Firewall Management

Command	What it does	Real-life Analogy
`ufw status`	Check firewall status (Ubuntu)	*Like checking if security guard is on duty*
`ufw enable`	Turn on the firewall	*Like hiring the security guard*
`ufw allow 22/tcp`	Allow SSH connections	*Open door 22 for staff (SSH)*
`ufw allow 80/tcp`	Allow HTTP web traffic	*Open door 80 for visitors (web)*
`ufw deny 3306`	Block MySQL port from outside	*Keep database door locked from outside*
`firewall-cmd --list-all`	Show firewall rules (RHEL)	*RHEL version of ufw status*

Sudoers File

```
# Edit sudoers safely (ALWAYS use visudo, not nano!)
$ sudo visudo

# Give alice permission to run ALL commands as root
alice  ALL=(ALL:ALL) ALL

# Give alice permission to restart nginx without password
alice  ALL=(ALL) NOPASSWD: /bin/systemctl restart nginx

# Give devs group sudo access
%devs  ALL=(ALL:ALL) ALL
```

Real-World Analogy: Building Security

UFW firewall = a security guard at the building entrance. Each 'ufw allow' rule = 'let in people with badge type X'. 'ufw deny 3306' = 'never let outsiders reach the server room (database)'. SSH keys = key cards that can't be duplicated. PermitRootLogin no = the CEO never uses the public entrance — always a private, secure route.

Chapter 16: Important DevOps Concepts in Linux

These are the day-to-day operational tasks every DevOps engineer must know. Cron jobs automate everything. Monitoring keeps servers healthy. Troubleshooting keeps you employed!

Cron Jobs — Automated Scheduling

```
Cron Syntax:  MIN  HOUR  DAY  MONTH  WEEKDAY  COMMAND
              |    |     |    |      |
              |    |     |    |      └── 0=Sun, 1=Mon...6=Sat
              |    |     |    └───────── 1-12
              |    |     └────────────── 1-31
              |    └──────────────────── 0-23
              └───────────────────────── 0-59

* = every possible value  (every minute, hour, day...)

Examples:
0 2 * * *    /backup.sh        → Every day at 2:00 AM
*/5 * * * *  /check_health.sh  → Every 5 minutes
0 0 * * 0    /cleanup.sh       → Every Sunday midnight
30 8 1 * *   /monthly.sh       → 1st of every month at 8:30
```

```
# Edit your cron jobs
$ crontab -e

# View current cron jobs
$ crontab -l

# Example: Backup database every day at 2 AM
0 2 * * * /opt/scripts/backup_db.sh >> /var/log/backup.log 2>&1

# Example: Clear temp files every Sunday at midnight
0 0 * * 0 rm -rf /tmp/* >> /var/log/cleanup.log 2>&1

# Example: Check disk space every 5 minutes
*/5 * * * * df -h | grep -E '9[0-9]%|100%' | mail -s "DISK ALERT"
admin@company.com
```

Port Troubleshooting

Command	What it does	Real-life Analogy
`ss -tulpn | grep :80`	Check what's on port 80	*Which program owns this door?*
`lsof -i :8080`	Show process using port 8080	*Who is blocking this port?*
`curl -v http://localhost`	Test local web server	*Test if your web server responds*
`telnet server 3306`	Test if port is reachable	*Like knocking on a specific door*

Command	What it does	Real-life Analogy
`nmap -p 1-1000 server`	Scan which ports are open	*Like checking all doors of a building*

Service Troubleshooting Workflow

```
Service not working? Follow this checklist:

1. systemctl status nginx     ← Is it running?
   ↓ (if failed)
2. journalctl -u nginx -n 50 ← What error stopped it?
   ↓ (check config error?)
3. nginx -t                   ← Is the config valid?
   ↓ (fix config, restart)
4. systemctl restart nginx
   ↓ (still failing?)
5. ss -tulpn | grep :80       ← Is port already in use?
   ↓ (port conflict)
6. lsof -i :80                ← Which process stole port 80?
   ↓
7. kill that process, restart nginx  ✓ Fixed!
```

Server Hardening Basics

Command	What it does	Real-life Analogy
`Disable root SSH`	Set PermitRootLogin no in sshd_config	*Never let CEO use public entrance*
`Use SSH keys only`	Set PasswordAuthentication no	*Key cards only, no PIN codes*
`Firewall default deny`	ufw default deny incoming	*Block all doors by default*
`Update regularly`	apt update && apt upgrade -y	*Always patch security holes*
`Fail2ban`	Auto-bans IPs with too many failed logins	*Security guard who bans repeat offenders*
`Least privilege`	Give users minimum needed permissions	*Staff can only access their own floor*

Log Rotation

Log files grow forever and will fill your disk. Log rotation automatically compresses old logs and deletes very old ones. It is configured in /etc/logrotate.conf and /etc/logrotate.d/.

```
# Example log rotation config (/etc/logrotate.d/nginx)
/var/log/nginx/*.log {
    daily               # rotate daily
    missingok           # don't error if log missing
    rotate 14           # keep 14 days of logs
    compress            # gzip old logs
    delaycompress       # don't compress yesterday's log yet
```

```
    notifempty          # don't rotate empty logs
    postrotate
        nginx -s reopen # tell nginx to use new log file
    endscript
}
```

Real-World Analogy: Cron Jobs

*Cron is like a reliable alarm clock for your server. Set it once and it never forgets. 'Every Sunday at midnight, clean the temp folder' = '0 0 * * 0 rm -rf /tmp/*'. Cron is the difference between a manual task you forget and an automated system that runs perfectly every time while you sleep. Every DevOps engineer automates with cron!*

— End of Linux Textbook —

tech__.talks